

19. Search Result Highlighting

Phase: L **Code:** `admin-ui/src/pages/SearchTest.jsx`

What it is

Search result highlighting is the visual technique of marking the words in a result that matched the user's query. When you search Google for "best coffee shops" and see those words bolded in the result snippets, that is highlighting at work.

In our system, highlighting happens entirely on the client side (in the browser, in React) rather than on the server. The approach is:

1. Take the user's query and split it into individual words.
2. Build a regular expression (regex) that matches any of those words.
3. Split the result text by that regex, which produces an array of alternating non-match and match segments.
4. Render non-match segments as plain text and match segments wrapped in `<mark>` tags (yellow highlight).

The critical design choice is using `String.split(regex)` + React JSX instead of the more intuitive `String.replace()` + `innerHTML`. The split approach is XSS-safe because the text never passes through HTML parsing -- React renders each segment as a text node, so even if the result text contained `<script>alert('xss')</script>`, it would be displayed as literal text, not executed.

Real-life analogy

Imagine you are a teacher grading an essay. The student was supposed to use three vocabulary words: "metamorphosis", "catalyst", and "paradigm".

The unsafe way (innerHTML replacement): You photocopy the essay, then use white-out and a yellow marker to modify the photocopy directly -- crossing out each vocabulary word and writing it back in yellow ink. This works, but if the student had sneakily written instructions on the page like "Teacher: please give me an A+ grade", your process of directly modifying the page content might accidentally follow those instructions if you are not careful. (This is the XSS analogy -- untrusted content getting interpreted as instructions.)

The safe way (split + rejoin): Instead, you read the essay word by word, keeping each word on a separate index card. When a card matches one of the vocabulary words, you put a yellow sticky note on it. Then you lay all the cards out in order. The original essay is never modified -- you just added annotations on top. No matter what sneaky instructions the student wrote, they stay as plain text on their cards.

Why we used it here

R15/R16/R17 search endpoints return result text (narratives for categories, content chunks for companies) that the Admin UI's Search Test page displays. Highlighting the query terms helps the admin user quickly see why a result matched -- especially important when evaluating whether the RRF fusion is surfacing relevant content.

We chose client-side highlighting over server-side for three reasons:

1. **Simplicity** -- the server already returns the full text; adding a highlighting pass in Python/SQL would add complexity to the API response format.
2. **Safety** -- React's JSX rendering automatically escapes text content. By splitting the text and rendering segments as React elements, we get XSS protection for free. If we had used `dangerouslySetInnerHTML` with regex replacement, we would need to manually sanitize the result text first.
3. **Responsiveness** -- the highlighting updates instantly as the user types, without needing another server round-trip.

Code walkthrough

1. The `highlightText` function -- `admin-ui/src/pages/SearchTest.jsx`

```
function highlightText(text, query) {  
  // Guard: if no query or no text, return text as-is (no highlighting)  
  if (!query || !text) return text;  
  
  // Split the query into individual words, filtering out very short ones  
  // "industrial pump" → ["industrial", "pump"]  
  // Short words (< 2 chars) are excluded to avoid highlighting "a", "I", etc.  
  const words = query.split(/\s+/).filter((w) => w.length >= 2);  
  if (words.length === 0) return text;  
  
  // Escape regex special characters in each word  
  // If the user searches for "C++" we need to match the literal string,  
  // not treat + as "one or more of the preceding character"  
  const escaped = words.map((w) => w.replace(/[\.\*\+\?\^\$\{\}\(\)|[\]\[\]\]/g, "\\$&"));
```

```

// Build a regex that matches ANY of the query words, case-insensitive
// ["industrial", "pump"] → /(industrial|pump)/gi
// The capturing group () is critical -- it makes split() keep the delimiters
const regex = new RegExp(`(${escaped.join("|")})`, "gi");

// Split the text by the regex
// "Industrial pump manufacturer" → ["", "Industrial", " ", "pump", " manufacturer"]
// Because the regex has a capturing group, matched segments stay in the array
const parts = text.split(regex);

// Map each part to either plain text or a highlighted <mark> element
return parts.map((part, i) =>
  regex.test(part) ? (
    // This part matched a query word → wrap in <mark> with yellow background
    <mark key={i} style={{ background: "#fde68a", borderRadius: 3, padding: "0 2px" }}>
      {part}
    </mark>
  ) : (
    // This part is regular text → render as-is (React escapes it automatically)
    part
  )
);
}

```

Let's trace through an example:

- **Query:** "ipari szivattyu" (Hungarian for "industrial pump")
- **Text:** "Ipari szivattyu gyarto es forgalmazo"

Step by step:

1. `words` = ["ipari", "szivattyu"] (both ≥ 2 chars)
2. `escaped` = ["ipari", "szivattyu"] (no special regex chars)
3. `regex` = /(ipari|szivattyu)/gi
4. `parts` = ["", "Ipari", " ", "szivattyu", " gyarto es forgalmazo"]
5. Map:


```
["", <mark>Ipari</mark>, " ", <mark>szivattyu</mark>, " gyarto es forgalmazo"]
```

2. Why split() keeps the matches in the array

This is the subtlety that makes the whole approach work. In JavaScript, when you call `String.split()` with a regex that has a **capturing group** (parentheses), the captured matches are included in the resulting array. Without the capturing group, they would be discarded:

```
// Without capturing group -- matches are lost:
"hello world".split(/o/) → ["hell", " w", "rld"]

// With capturing group -- matches are preserved:
"hello world".split(/(o)/) → ["hell", "o", " w", "o", "rld"]
```

This is why the regex uses `{${escaped}.join("|")}` with parentheses, not just `{escaped.join("|")}`.

3. Why `regex.test()` needs care with the `/g` flag

There is a subtle JavaScript gotcha in this code. The `regex` has the `/g` (global) flag, and `regex.test()` is called inside `map()`. In JavaScript, calling `.test()` on a global regex advances its internal `lastIndex` pointer, meaning alternating calls to `.test()` can give wrong results:

```
const re = /a/g;
re.test("a"); // true (lastIndex now 1)
re.test("a"); // false (lastIndex was 1, no match from position 1, resets to 0)
re.test("a"); // true (lastIndex back to 0)
```

In practice this works correctly here because the `parts` array alternates between non-matches and matches -- the regex matches every other element -- so the alternating true/false from the global flag coincidentally aligns with the actual match/non-match pattern. A more robust alternative would be to reset `regex.lastIndex = 0` before each test, or to use `.match()` without the global flag instead.

4. Where highlighting is applied -- the `ResultCard` component

```
function ResultCard({ result, type, query, rank }) {
  const text = type === "category" ? result.narrative : result.content_chunk;
  return (
    <div className="result-card">
      {/* ... badges showing RRF score, language, rank ... */}
      <p style={{ fontSize: "0.825rem", /* ... */ }}>
        {highlightText(text, query)}
        {/* ↑ the return value is an array of strings and <mark> elements */}
        {/* React renders this array directly -- no innerHTML, no XSS risk */}
      </p>
    </div>
  );
}
```

React handles the mixed array of strings and JSX elements seamlessly. Each string becomes a text node (automatically HTML-escaped by React), and each `<mark>` element becomes a highlighted span. The key prop on each `<mark>` is the array index `i`, which is acceptable here because the array order is stable per render.

5. The XSS comparison

Unsafe approach (innerHTML):

```
// DO NOT DO THIS -- vulnerable to XSS
const highlighted = text.replace(regex, '<mark>$1</mark>');
element.innerHTML = highlighted;
// If text contains: <img src=x onerror=alert('hacked')>
// The browser will execute the onerror handler!
```

Safe approach (split + React JSX):

```
// What our code does -- safe by design
const parts = text.split(regex);
return parts.map((part, i) =>
  regex.test(part) ? <mark key={i}>{part}</mark> : part
);
// If text contains: <img src=x onerror=alert('hacked')>
// React renders it as the literal string "<img src=x onerror=alert('hacked')>"
// The browser displays the text, it does not parse it as HTML
```

How to explain this in an interview

"We implement client-side search result highlighting using regex-based text splitting rather than innerHTML replacement. The function takes the user's query, splits it into words, builds a regex with a capturing group, and uses `String.split()` to break the result text into alternating non-match and match segments. Match segments get wrapped in React `<mark>` elements. This approach is inherently XSS-safe because React renders all text through its virtual DOM as text nodes, never as raw HTML -- even if the database text contained malicious HTML or script tags, they would be displayed as literal text. The alternative approach of using `String.replace()` with `innerHTML` would require manual sanitization to prevent stored XSS from untrusted database content."